Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 9: **Pengolahan Data: Import, Export, dan Queue**

Dari pipeline ETL, dataset besar, dan memory, sampai pekerjaan berat yang dititipkan ke background

Yang Akan Kamu Pelajari

1. Menjelaskan pola **ETL** dan **batch processing** untuk memproses data dalam jumlah besar
2. Menyusun laporan rentang tanggal, dan memilih teknik yang tepat untuk **dataset besar**
3. Memahami kenapa **memory** bisa habis, dan memilih cara membaca data yang hemat memory
4. Memilih format **CSV, Excel, atau PDF** sesuai kebutuhan
5. Mengikuti perjalanan sebuah **job** di queue, dari dibuat sampai dieksekusi worker, dan menjalankannya di **production**

> Slide ini membahas konsep. Menerapkan laporan, export/import, dan queue pada Simple POS dikerjakan di jobsheet.

Peta Pertemuan

Data mentah → **ETL + Batch** → **Laporan** → **CSV / Excel / PDF** → **Queue**

> Seperti dapur restoran: bahan masuk (**extract**), dimasak (**transform**), lalu disajikan (**load**). Kalau pesanan membludak, tamu diberi nomor meja dan hidangan diantar belakangan. Itu queue.

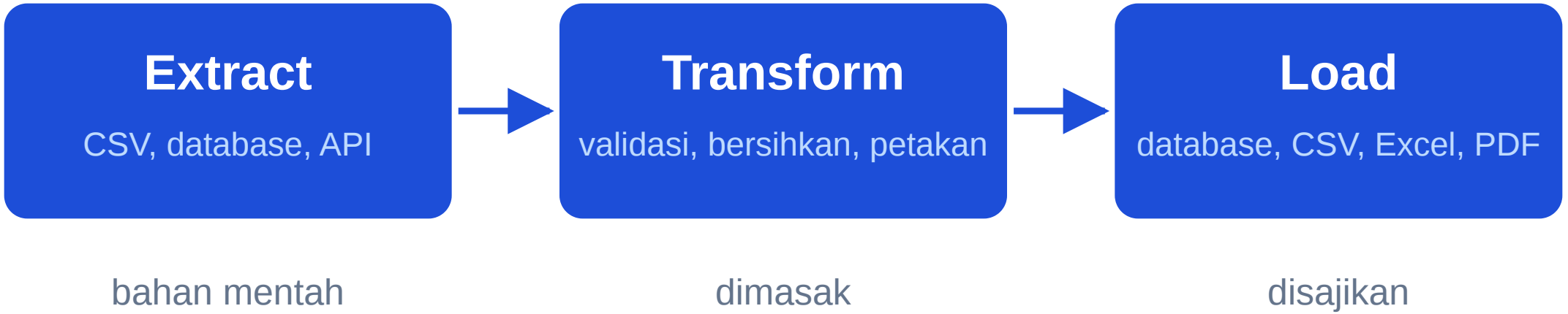
Bagian 1

ETL dan Batch Processing

Empat Istilah Dasar

Istilah	Artinya
ETL	Extract (ambil), Transform (ubah), Load (simpan atau sajikan)
Batch processing	memproses data berkelompok, bukan satu per satu
Import / Export	data masuk dari CSV/Excel ke database, atau keluar ke CSV/Excel/PDF
Queue	pekerjaan berat dititipkan, dikerjakan proses lain di background

ETL: Tiga Tahap yang Selalu Muncul



- Pisahkan tiga tahap ini supaya tiap bagian mudah diuji dan diganti
- Sumber, aturan validasi, dan tujuan boleh berubah tanpa menyentuh bagian lain

Import dan Export Adalah ETL Dua Arah

	Extract	Transform	Load
Import	baca file CSV / Excel	validasi tiap row, petakan column	simpan ke database
Export	query database	pilih column, format tanggal dan angka	tulis CSV / Excel / PDF

> Untuk setiap fitur, tanyakan: dari mana datanya, apa yang diubah, ke mana hasilnya?

Batch: Satu per Satu vs Satu Truk

```php
// 10.000 row = 10.000 query
foreach ($rows as $row) {
    Article::create($row);
}

// 10.000 row = 20 query (batch 500 row)
foreach (array_chunk($rows, 500) as $batch) {
    Article::insert($batch);
}
```

Setiap query punya ongkos perjalanan ke database. Satu truk penuh lebih hemat daripada 10.000 kurir motor.

> `insert()` massal tidak mengisi `created_at` / `updated_at` dan tidak menjalankan model event. Isi timestamp sendiri kalau perlu.

Ukuran Batch dan Membaca per Batch

Terlalu kecil

- Terlalu banyak round trip
- Lambat

Terlalu besar

- Memory membengkak
- Satu kegagalan menggagalkan banyak row

```php
Article::where('status', 'draft')
    ->chunkById(500, function ($articles) {
        foreach ($articles as $a) {
            // proses tiap row
        }
    });
```

> Mulai dari ratusan sampai seribu row per batch, lalu ukur. Bungkus tiap batch dengan `DB::transaction()` kalau harus berhasil atau gagal bersama.

Idempotent: Aman Dijalankan Berkali-kali

File import yang sama di-upload dua kali tidak boleh membuat data dobel.

```
Article::updateOrCreate(
    ['slug' => $row['slug']],     // kunci pencarian
    ['title' => $row['title']],   // nilai yang diperbarui
);

// versi batch: satu query untuk banyak row
Article::upsert($batch, ['slug'], ['title']);
```

> `upsert()` butuh **unique index** pada column kuncinya. Tanpa itu, database tidak tahu row mana yang dianggap sama.

Bagian 2

Laporan dan Dataset Besar

Daftar vs Laporan

- **Daftar**: menampilkan row satu per satu
- **Laporan**: merekap, misalnya "total X pada rentang tanggal tertentu"

```
$totalKata = Article::whereBetween('published_at', [$from, $to])
    ->sum('word_count');   // filter tanggal, lalu jumlahkan satu column
```

Jebakan Umum: Batas Hari yang Terpotong

> Filter "1 sampai 31 Agustus" berarti "sampai 31 Agustus pukul 00:00:00". Seluruh transaksi tanggal 31 ikut hilang.

```
$from = $request->date('from')->startOfDay();   // 00:00:00
$to = $request->date('to')->endOfDay();         // 23:59:59
```

Jebakan Batas Hari, Digambar di Garis Waktu



Hitung di Database, Bukan di PHP

```php
// berat: semua row dimuat ke memory, dijumlahkan di PHP
$total = Article::all()->sum('word_count');

// ringan: database yang menjumlahkan
$total = Article::sum('word_count');

// rekap per author: 5 author dengan kata terbanyak
Article::selectRaw('author_id, COUNT(*) AS jumlah, SUM(word_count) AS total_kata')
    ->groupBy('author_id')
    ->orderByDesc('total_kata')
    ->limit(5)
    ->get();
```

> Seperti menanyakan saldo ke bank, bukan meminta seluruh mutasi lalu menjumlahkannya sendiri.

Pakai Total yang Sudah Tersimpan

```
// word_count dihitung sekali saat artikel disimpan
$article->word_count = str_word_count($article->body);

// laporan cukup menjumlahkan column yang sudah tersimpan
Article::whereBetween('published_at', [$from, $to])->sum('word_count');
```

- Nilai turunan dihitung di server saat data dibuat (Pertemuan 6)
- Laporan tidak perlu menghitung ulang dari data mentah tiap kali dibuka

Perkakas untuk Dataset Besar

Teknik	Dipakai saat	Catatan
`paginate()`	menampilkan daftar	satu halaman per request
`chunkById()`	meng-update banyak row	aman saat row berubah
`lazyById()`	membaca banyak row seperti `foreach`	memory datar
`cursor()`	satu query panjang	kurang aman kalau data berubah
`select([...])`	hanya butuh sebagian column	model lebih kecil
`with()`	memakai relasi di loop	menghindari N+1
index	filter atau urut pada column	tidak scan seluruh tabel

Index: Daftar Isi untuk Tabel

```
Schema::table('articles', function (Blueprint $table) {
    $table->index('published_at');
});
```

```
EXPLAIN SELECT * FROM articles WHERE published_at BETWEEN '2026-08-01' AND '2026-08-31';
-- type: ALL    = scan seluruh tabel (lambat)
-- type: range  = memakai index (cepat)
```

Tanpa index, database membaca tabel row demi row. Dengan index, ia langsung lompat ke row yang dicari.

> Mengingat Pertemuan 4: strategi index dan `EXPLAIN`

Bagian 3

Memory Management

Kenapa Memory Bisa Habis

- PHP punya batas `memory_limit` (umumnya 128 MB)
- Tiap row Eloquent menjadi **object model**, jauh lebih besar dari data mentahnya
- `get()` memuat semua row sekaligus

> Contoh di jobsheet: membaca 52.500 transaksi dengan `get()` memakai sekitar 96 MB, sedangkan `lazyById()` hanya sekitar 38 MB. Tambah datanya sedikit lagi dan `get()` kena error *Allowed memory size exhausted*.

`get` vs `chunk` vs `lazy` vs `cursor`

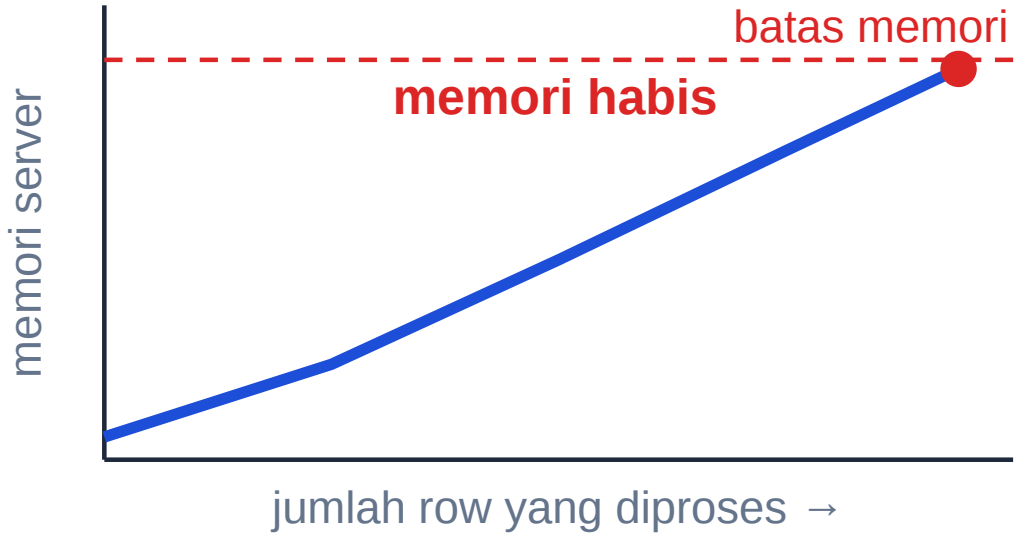
Cara	Memory	Query	Cocok untuk
`get()`	semua row	1	data kecil
`chunkById(n)`	n row	banyak	update massal
`lazyById(n)`	n row	banyak	export, `foreach` biasa
`cursor()`	1 row	1	baca sekali, data tidak berubah

> Data bisa ratusan ribu row? Jangan `get()`. Baca dengan `lazyById()`, update dengan `chunkById()`.

Ember vs Keran

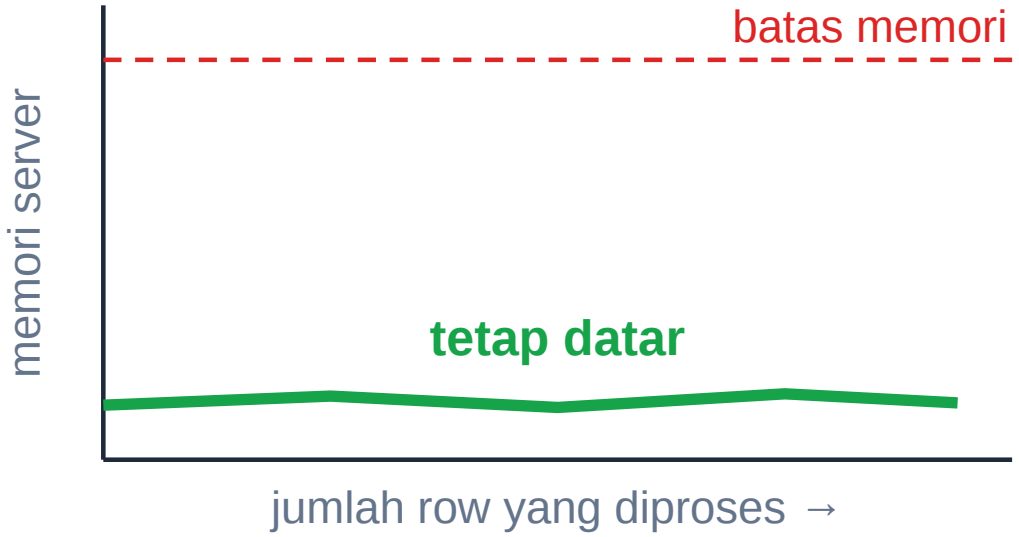
Ember: tampung dulu

get() lalu susun satu string besar



Keran: alirkan per row

lazyById() + streamDownload()



Export dengan `streamDownload()` dan `lazyById()`

```php
$articles = Article::with('author')
    ->whereBetween('published_at', [$from, $to]);

return response()->streamDownload(function () use ($articles) {
    $out = fopen('php://output', 'w');
    fputcsv($out, ['Judul', 'Penulis', 'Tanggal']);
    foreach ($articles->lazyById() as $a) {
        fputcsv($out, [$a->title, $a->author->name, $a->published_at]);
    }
    fclose($out);
}, 'laporan-artikel.csv');
```

`lazyById()` membaca per chunk, `streamDownload()` mengirim per row. Memory tetap datar.

> Laravel docs: `HTTP Responses → File Downloads` dan `Eloquent → Chunking`

Jangan Menebak, Ukur

```php
gc_collect_cycles();
memory_reset_peak_usage();      // mulai hitung peak dari nol

foreach (Article::lazyById() as $a) {
    // ... pekerjaan ...
}

printf("peak %.1f MB\n", memory_get_peak_usage() / 1048576);
```

- `memory_get_peak_usage()` : titik tertinggi pemakaian memory
- `memory_reset_peak_usage()` (PHP 8.2+): tiap pengukuran berdiri sendiri
- Bandingkan dua cara pada data yang sama, lalu catat angkanya

Jebakan `chunk()`: Row yang Terlewat

```
// BERBAHAYA: filter 'draft', lalu mengubah 'draft' itu sendiri
Article::where('status', 'draft')->chunk(100, function ($articles) {
    foreach ($articles as $a) {
        $a->update(['status' => 'published']);
    }
});
```

Row yang di-update keluar dari filter, daftar bergeser, dan chunk berikutnya melompati sebagian row.

> Solusi: `chunkById(100, ...)`. Ia memakai `id` terakhir sebagai penanda, jadi aman walau row berubah.

Bagian 4

Format dan Library: CSV, Excel, PDF

Memilih Format: CSV, Excel, atau PDF

	CSV	Excel (.xlsx)	PDF
Cocok untuk	data mentah, antar sistem	analisis, laporan keuangan	dibaca atau dicetak
Memory	sangat hemat (stream)	sedang	berat (seluruh dokumen)
Library	`fputcsv` (bawaan PHP)	Laravel Excel	DomPDF
Data besar	ya	ya (`FromQuery`)	tidak, batasi row

CSV: Sederhana tapi Penuh Jebakan

- **Delimiter**: Excel versi Indonesia sering memakai titik koma (`;`)
- **Quoting**: nilai berisi koma atau baris baru harus diberi tanda kutip (`fputcsv` mengurusnya)
- **Encoding**: UTF-8 dari Excel sering diawali karakter tak terlihat (**BOM**) yang merusak header pertama
- **Tanpa tipe**: semua nilai teks, validasi tipe tetap tugasmu

> Semua row ditolak saat import? Cek header dulu: delimiter dan BOM tersangka utama.

Laravel Excel: Export

```
class ArticlesExport implements FromQuery, WithHeadings, WithMapping, ShouldAutoSize
{
    public function query()
    {
        return Article::with('author')->orderBy('id');
    }

    public function headings(): array { return ['Judul', 'Penulis', 'Tanggal']; }

    public function map($article): array
    {
        return [$article->title, $article->author->name, $article->published_at];
    }
}

return Excel::download(new ArticlesExport, 'artikel.xlsx');
```

Kamu cukup mendeskripsikan isinya lewat interface kecil (`concern`). Format `.xlsx` diurus library.

> docs.laravel-excel.com → Exports

Laravel Excel: Import

```php
class ArticlesImport implements ToModel, WithHeadingRow, WithValidation,
    WithChunkReading, WithBatchInserts
{
    public function model(array $row): Article
    {
        return new Article(['title' => $row['title'], 'author_id' => $row['author_id']]);
    }

    public function rules(): array
    {
        return ['title' => ['required'], 'author_id' => ['exists:authors,id']];
    }

    public function chunkSize(): int { return 1000; }   // baca 1000 row per giliran
    public function batchSize(): int { return 500; }    // insert 500 row per query
}

Excel::import(new ArticlesImport, $request->file('file'));
```

Default-nya, `WithValidation` berhenti di row cacat pertama. Tambahkan `SkipsOnFailure` untuk melewati dan mengumpulkan row cacat.

PDF dengan DomPDF: HTML Jadi Dokumen

View Blade → HTML + CSS → DomPDF → File PDF

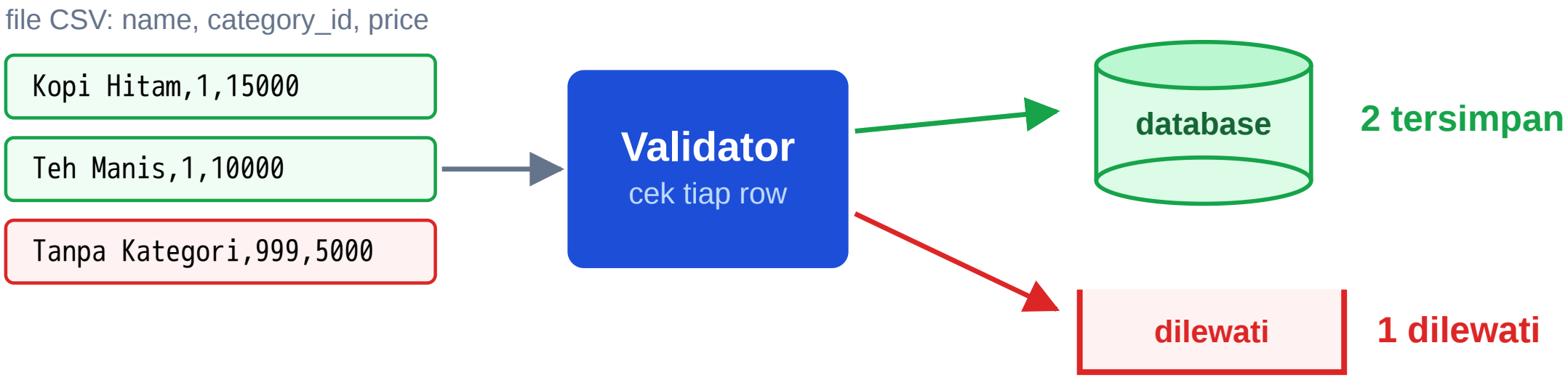
```
$pdf = Pdf::loadView('reports.articles', ['articles' => $articles])
    ->setPaper('a4', 'landscape');   // tabel lebar: pakai landscape

return $pdf->download('artikel.pdf');   // stream() untuk tampil di browser
```

DomPDF hanya mendukung sebagian CSS dan memuat seluruh dokumen di memory. Pakai untuk ringkasan kecil. Data besar: pilih CSV atau Excel.

Import: Satu Row Cacat Tidak Menghentikan yang Lain

Seperti kasir memindai belanjaan: barcode rusak disisihkan, sisanya tetap dipindai.



satu row cacat tidak menghentikan row lain

Import: Validasi Tiap Row, Lewati yang Cacat

```php
$rows = file($path, FILE_IGNORE_NEW_LINES | FILE_SKIP_EMPTY_LINES);
$rows = array_map('str_getcsv', $rows);
$header = array_shift($rows);

foreach ($rows as $row) {
    $data = array_combine($header, $row);

    $validator = Validator::make($data, [
        'slug' => ['required', 'string'],
        'title' => ['required', 'string'],
        'author_id' => ['required', 'exists:authors,id'],
    ]);

    if ($validator->fails()) {
        continue; // lewati row ini, lanjut ke row berikutnya
    }

    Article::updateOrCreate(['slug' => $data['slug']], $validator->validated());
}
```

Apa yang Dilakukan dengan Row Cacat?

Strategi	Cara kerja	Cocok untuk
Skip tanpa jejak	dilewati tanpa catatan	hampir tidak pernah: data hilang diam-diam
Skip dan catat	dilewati, nomor row dan alasan disimpan	import umum (disarankan)
Abort semua	satu row cacat membatalkan semuanya (`DB::transaction`)	data keuangan atau yang harus utuh

Bagian 5

Queue: Perjalanan Sebuah Job

Kapan Pekerjaan Masuk Queue?

Langsung (dalam request)

- Pengguna menunggu sampai selesai
- Cocok untuk pekerjaan singkat

Lewat queue (job)

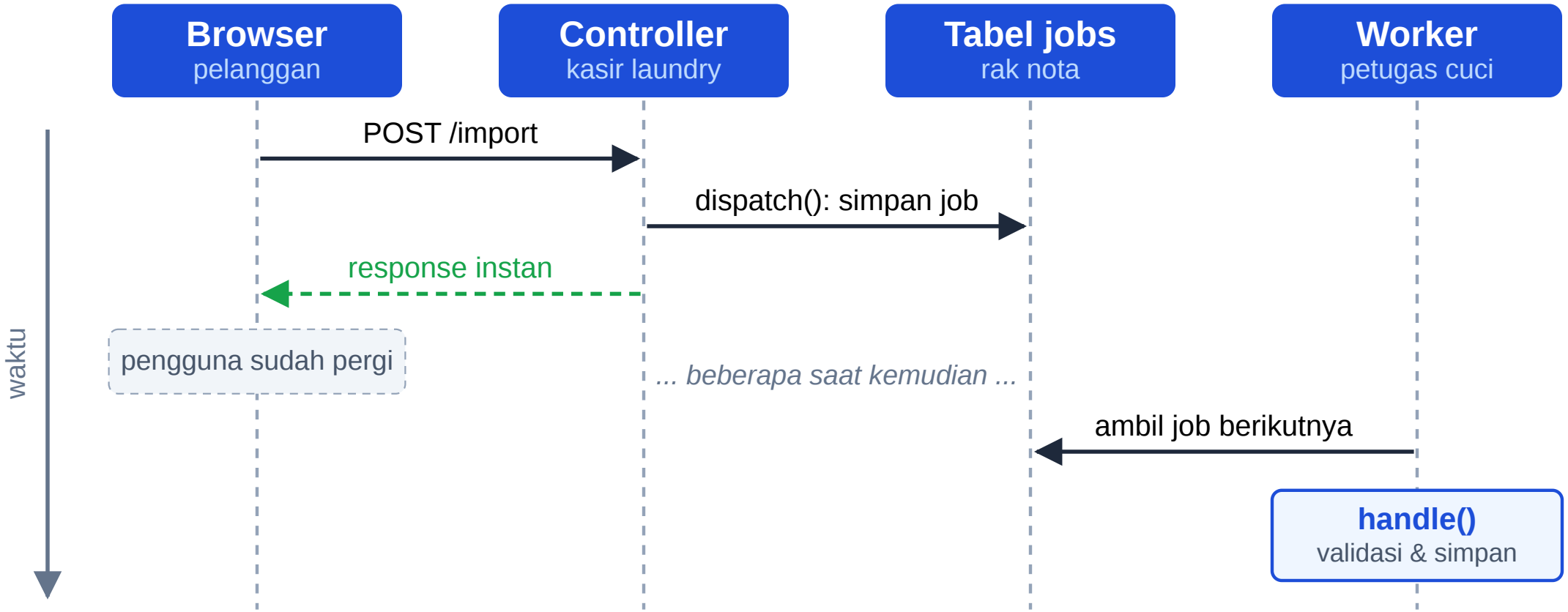
- Response instan, kerja di background
- Cocok untuk pekerjaan berat

Layak masuk queue kalau:

- durasinya tidak terprediksi (tergantung ukuran data)
- hasilnya tidak perlu terlihat seketika
- memanggil layanan lambat (email, API pihak ketiga)
- aman dicoba ulang (idempotent)

> Laundry: satu kaus ditunggu, satu karung dititip dengan nota. HTTP request punya batas waktu tunggu.

Queue = Laundry Kiloan



Satu Contoh dari Awal sampai Akhir

Kita ikuti satu pekerjaan: **import artikel dari CSV**.



> Lima langkah berikut memakai satu job yang sama: `ProcessArticleImport`.

Langkah 1: Buat Class Job

```
php artisan make:job ProcessArticleImport
```

```php
class ProcessArticleImport implements ShouldQueue
{
    use Queueable;

    public int $tries = 3;                // maksimal 3 attempt
    public array $backoff = [10, 60];     // jeda 10 detik, lalu 60 detik
    public int $timeout = 60;             // batalkan kalau lebih dari 60 detik

    public function __construct(public string $path) {}   // bawa path, bukan isi file

    public function handle(): void
    {
        $rows = array_map('str_getcsv', file(Storage::path($this->path), FILE_IGNORE_NEW_LINES));
        $header = array_shift($rows);

        foreach ($rows as $row) {
            $data = array_combine($header, $row);
            Article::updateOrCreate(['slug' => $data['slug']], ['title' => $data['title']]);
        }
    }
}
```

Langkah 2: Controller Mengirim Job

```php
public function store(Request $request)
{
    $request->validate(['file' => ['required', 'file', 'mimes:csv,txt']]);

    $path = $request->file('file')->store('imports');   // simpan file dulu

    ProcessArticleImport::dispatch($path);               // titip job ke queue

    return back()->with('success', 'Import sedang diproses di background.');
}
```

- Controller hanya menyimpan file dan mengirim job, lalu langsung response
- Loop validasi dan simpan pindah ke `handle()`, **belum berjalan**

Langkah 3: Job Menunggu di Tabel `jobs`

`dispatch()` hanya menulis satu row. Belum ada artikel yang diimport.

id	queue	payload (ringkas)	attempts	reserved_at
1	default	`{"displayName":"App\\Jobs\\ProcessArticleImport", ... "path":"imports/abc.csv"}`	0	NULL

- `payload` : nama class job dan data yang dibawa (path file)
- `attempts = 0` : belum pernah dicoba
- `reserved_at = NULL` : belum diambil worker

> Tanpa worker, row ini menumpuk. Nota bertambah, tapi tidak ada cucian yang selesai.

Langkah 4: Worker Mengeksekusi Job

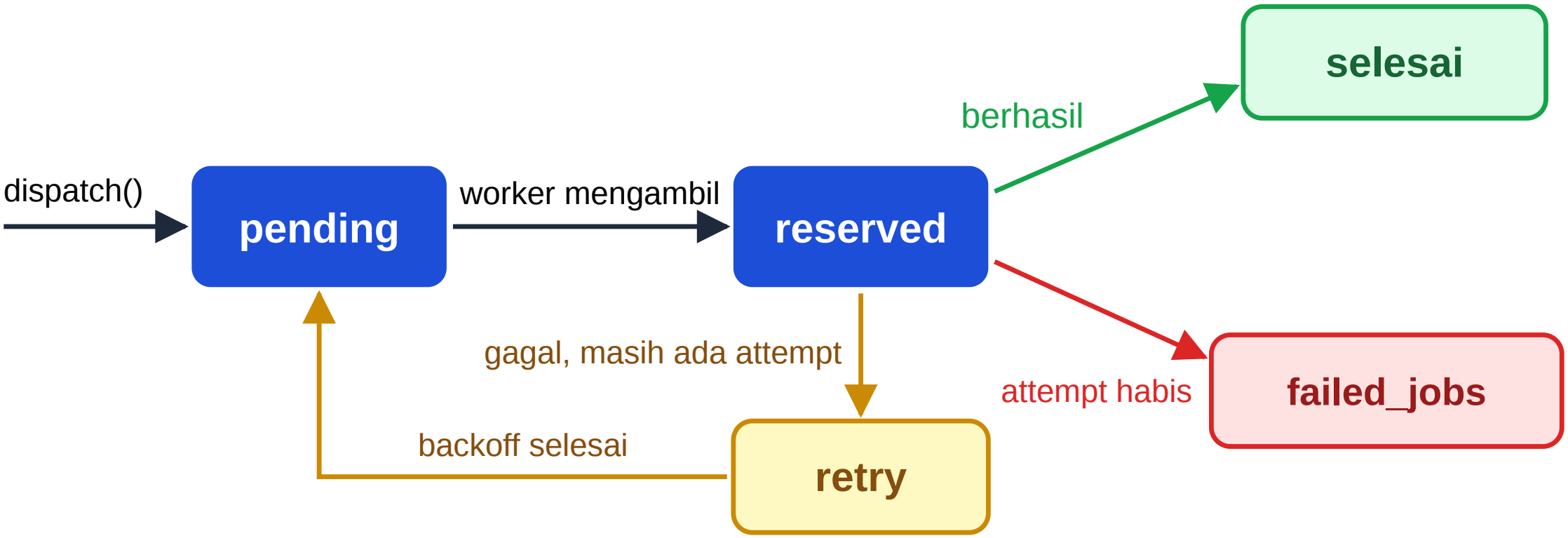
```
php artisan queue:work
```

```
  2026-10-21 10:00:00 App\Jobs\ProcessArticleImport ........ RUNNING
  2026-10-21 10:00:07 App\Jobs\ProcessArticleImport ... 7s DONE
```

1. Worker mengambil row dari `jobs`, mengisi `reserved_at`, `attempts` jadi 1
2. Worker membuat object job dari `payload`, lalu memanggil `handle()`
3. Berhasil: row dihapus dari `jobs`. Artikel sekarang ada di database

> **Worker:** proses PHP yang terus berjalan, mengambil job satu per satu, lalu menjalankan `handle()`-nya.

Langkah 5: Kalau Job Gagal




```
$tries = 3, $backoff = [10, 60]
```

Langkah 5: Gagal, Retry, lalu `failed_jobs`

```
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 1, tunggu 10 detik
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 2, tunggu 60 detik
  App\Jobs\ProcessArticleImport ........ RUNNING
  App\Jobs\ProcessArticleImport ... 1s FAIL     <- attempt 3 = $tries, masuk failed_jobs
```

```
php artisan queue:failed          # lihat job yang gagal, lengkap dengan exception
php artisan queue:retry all       # jalankan ulang setelah penyebabnya diperbaiki
php artisan queue:flush           # hapus catatan job gagal
```

Rangkuman Alur Satu Job



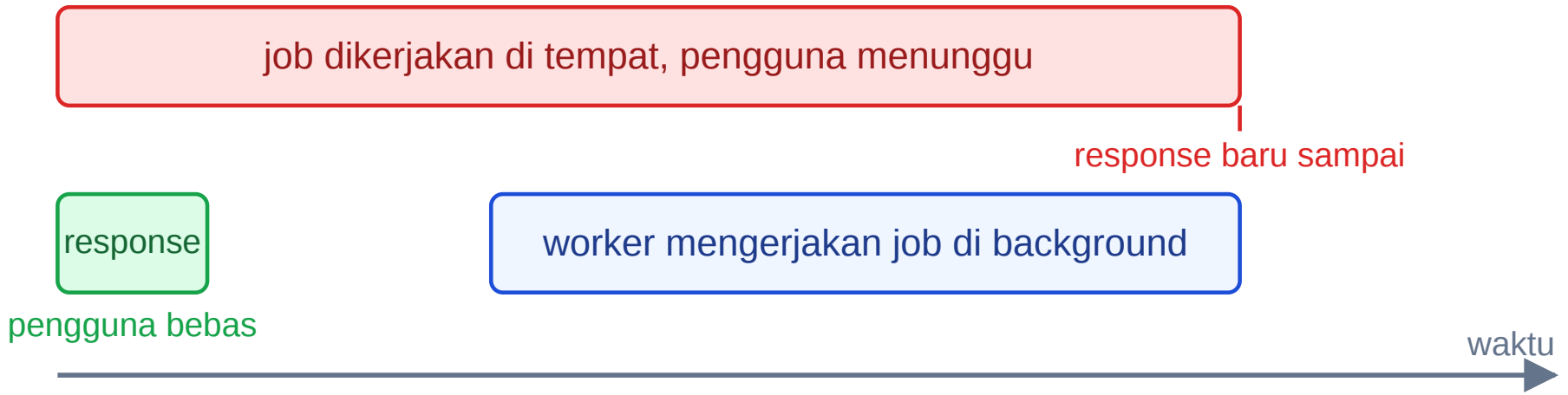
Berhasil

- row `jobs` dihapus
- data sudah tersimpan

Gagal

- retry sesuai `$tries` dan `$backoff`
- habis: pindah ke `failed_jobs`

`sync` vs `database`: Siapa yang Menunggu?



> Kalau `QUEUE_CONNECTION=sync` di `.env`, job langsung jalan di dalam request. Tidak ada queue sungguhan.

Aturan Merancang Job

Lakukan

- Rancang **idempotent** (`updateOrCreate`)
- Bawa ID atau path, bukan data besar
- Atur `$tries`, `$backoff`, `$timeout`
- Pakai `->afterCommit()` kalau job bergantung pada data di transaksi

Hindari

- `request()`, session, atau user login di dalam `handle()`: di background semuanya tidak ada
- Isi file di payload job
- Mengandalkan urutan job tanpa dirancang
- Membiarkan job gagal tanpa pemantauan

Memberi Kabar ke Pengguna

- **Column status** (`pending` , `processing` , `done` , `failed`) yang di-update job dan ditampilkan di halaman
- **Notifikasi** (email atau in-app) saat job selesai atau gagal
- **Job batch**: `Bus::batch($jobs)->dispatch()` melacak banyak job, `$batch->progress()` memberi persentase

> Laravel docs: `Queues → Job Batching`

Bagian 6

Queue di Production

Persiapan: `.env` dan Tabel Queue

```
# .env
QUEUE_CONNECTION=database
```

Driver	Penyimpanan	Cocok untuk
`sync`	tidak ada, job jalan di tempat	development, testing
`database`	tabel `jobs`	proyek kecil sampai menengah
`redis`	Redis (in-memory)	trafik tinggi

- Proyek Laravel baru sudah punya migration tabel `jobs` dan `failed_jobs`
- Proyek lama: `php artisan make:queue-table`, lalu `php artisan migrate`
- Setelah mengubah `.env`: `php artisan config:clear` (atau `config:cache` di production)

`retry_after` vs `--timeout`

```
// config/queue.php
'database' => [
    'driver' => 'database',
    'retry_after' => 90,    // setelah 90 detik, job dianggap macet dan diambil worker lain
],
```

Benar

- `--timeout=60`
- `retry_after = 90`
- job dihentikan sebelum dianggap macet

Salah

- `--timeout=120`
- `retry_after = 90`
- job masih jalan, tapi diambil worker kedua: **diproses dobel**

> Aturan: `timeout` harus lebih kecil dari `retry_after`.

Opsi `queue:work` yang Perlu Dikenal

Opsi	Fungsi
`--queue=high,default`	proses queue `high` dulu, baru `default`
`--tries=3`	maksimal attempt per job
`--timeout=60`	batas detik per job
`--sleep=3`	jeda detik saat queue kosong
`--max-jobs=1000`	berhenti setelah n job (memory dibersihkan)
`--max-time=3600`	berhenti setelah n detik
`--stop-when-empty`	berhenti kalau queue kosong

> Worker yang berhenti sendiri (`--max-time` , `--max-jobs`) akan dinyalakan lagi oleh Supervisor, sehingga memory tidak menumpuk.

Supervisor: Menjaga Worker Tetap Hidup

`queue:work` di terminal mati saat terminal ditutup atau server restart. Supervisor menyalakannya lagi otomatis.

```
; /etc/supervisor/conf.d/laravel-worker.conf
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/app/artisan queue:work --sleep=3 --tries=3 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=2
redirect_stderr=true
stdout_logfile=/var/www/app/storage/logs/worker.log
stopwaitsecs=3600
```

Mengaktifkan Supervisor

```
sudo apt-get install supervisor

sudo supervisorctl reread                       # baca file konfigurasi baru
sudo supervisorctl update                       # terapkan
sudo supervisorctl start "laravel-worker:*"     # nyalakan semua worker
```

- `numprocs=2` : dua worker berjalan bersamaan
- `autorestart=true` : worker mati, Supervisor menyalakannya lagi
- `stopwaitsecs` : harus lebih besar dari job terlama, supaya job tidak terpotong

> Laravel docs: `Queues → Supervisor Configuration`

Deploy: Restart Worker

Worker hidup lama dan menyimpan kode lama di memory. Setelah deploy, kode baru tidak terbaca sampai worker restart.

```
git pull
composer install --no-dev --optimize-autoloader
php artisan migrate --force
php artisan config:cache
php artisan queue:restart     # worker berhenti setelah job yang berjalan selesai
```

> `queue:restart` tidak memotong job yang sedang jalan. Worker berhenti dengan rapi, lalu Supervisor menyalakannya dengan kode baru.

Naik Kelas: Redis dan Horizon

- Mulai dari `database`. Pindah ke `redis` kalau trafik tinggi atau job sangat banyak
- Ganti `QUEUE_CONNECTION=redis` dan siapkan server Redis
- **Horizon**: dashboard dan konfigurasi untuk queue Redis (metrik, job gagal, jumlah worker otomatis)
- Horizon hanya untuk Redis, dan dijaga Supervisor juga (`php artisan horizon`)

> Laravel docs: `Horizon` dan `Queues → Supervisor Configuration`

Menerapkan pada Simple POS

Di jobsheet, kamu menerapkan konsep ini pada Simple POS:

1. Laporan penjualan dengan filter tanggal, dihitung di database
2. Export laporan ke CSV (streaming), Excel, dan PDF
3. Mengukur memory `get()` vs `lazyById()` pada 50.000 row
4. Import produk dari CSV yang validasi per row dan aman diulang
5. Memindahkan import ke queued job dengan `$tries` dan `backoff`, lalu menelusuri job yang gagal

> Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Rangkuman (1/4): ETL, Batch, dan Laporan

- ETL: extract, transform, load. Import dan export adalah ETL dua arah
- Batch mengurangi round trip (`array_chunk` + `insert`, `chunkById`)
- Import idempotent: `updateOrCreate()` atau `upsert()` (dengan unique index)
- Laporan: `whereBetween`, `startOfDay()`/`endOfDay()`, hitung di database, tambah index

Rangkuman (2/4): Memory dan Format

- `get()` memuat semua row. Baca dengan `lazyById()`, update dengan `chunkById()`
- Ukur dengan `memory_get_peak_usage()`, jangan menebak
- CSV untuk data mentah, Excel untuk analisis, PDF untuk dokumen ringkas
- Row cacat saat import: skip dan catat, atau abort untuk data yang harus utuh

Rangkuman (3/4): Perjalanan Job

- `make:job`, lalu `dispatch()` hanya menulis row ke tabel `jobs`
- `queue:work` mengambil row dan menjalankan `handle()`
- Gagal: retry sesuai `$tries` dan `$backoff`, habis masuk `failed_jobs`, lalu `queue:retry`
- Job idempotent, bawa ID atau path, jangan bergantung pada request atau session

Rangkuman (4/4): Queue di Production

- `QUEUE_CONNECTION=database` (atau `redis`), tabel `jobs` lewat migration
- `timeout` harus lebih kecil dari `retry_after`
- Supervisor menjaga `queue:work` tetap hidup (`numprocs`, `autorestart`, `stopwaitsecs`)
- Deploy: `queue:restart`. Skala besar: Redis dan Horizon

Referensi & Diskusi

Dokumentasi resmi Laravel (Queues, Horizon, File Storage, HTTP Responses, Eloquent Chunking), docs.laravel-excel.com, dan laravel-dompdf

Kode lengkap: `github.com/se-polinema/simple-pos-ch08`

Pertemuan berikutnya: Merancang dan Membangun REST API